

Job Assignment Using Network Flow

Joshua Straus

November 2025

Revised September 2026

1 Problem Description

In order to make roads safe next winter, the city of Madison is planning work shifts for snow plough drivers. There are 6 shifts, numbered 1 through 6, of 4 hours each in a day, and a total of n jobs over all 6 shifts combined. The i -th job belongs to shift $s_i \in [6]$ and needs to be assigned exactly d_i drivers. There are m drivers in total. Due to the characteristics of their snow ploughs and other constraints, the j -th driver can only help with jobs in $R_j \subseteq [n]$. Moreover, each driver can work at most 2 shifts per day, and during a given work shift can help with only a single job.

Design an algorithm that takes as input $n, m, (s_i, d_i)$ for $i \in [n]$ and R_j for $j \in [m]$, and outputs a daily assignment of jobs for each driver such that all requirements are met, or reports that no such assignment exists. Your algorithm should run in time $O(N^3)$, where $N := n + m$.

Solution

To solve this problem, we will create an algorithm that:

1. Creates a flow network representing the problem.
2. Runs a maximum-flow algorithm on the network.
3. Uses the resulting flow to produce a valid assignment, if one exists.

First, we construct the flow network. In this network, each unit of flow along an s - t path will represent the assignment of one driver to one job.

Create a set D of m vertices, with one vertex D_j for each driver j . Create a set J of n vertices, with one vertex J_i for each job i . Let K be a set of $6m$ vertices, where $K_{j,\ell}$ represents driver j working during shift ℓ , for $\ell \in \{1, \dots, 6\}$. Finally, let s be the source vertex and t be the sink vertex.

Connect s to each driver vertex D_j with an edge of capacity 2. This represents the fact that each driver may be assigned to at most two jobs.

For each driver j and shift ℓ , connect D_j to $K_{j,\ell}$ with an edge of capacity 1 if R_j contains at least one job occurring during shift ℓ . This capacity ensures that driver j can perform at most one job during shift ℓ .

Next, connect $K_{j,\ell}$ to job vertex J_i with an edge of capacity 1 if job i occurs during shift ℓ and $i \in R_j$. Thus, such an edge exists exactly when driver j is eligible to perform job i during its assigned shift.

Finally, connect each job vertex J_i to t with an edge of capacity d_i , where d_i is the number of drivers required for job i .

Run a maximum-flow algorithm on this network. Let

$$D_{\text{required}} = \sum_{i=1}^n d_i.$$

If the maximum-flow value is D_{required} , output the assignments represented by the flow: for every edge $K_{j,\ell} \rightarrow J_i$ carrying one unit of flow, assign driver j to job i . If the maximum-flow value is less than D_{required} , output that no valid assignment exists.

Time Complexity

The graph contains

$$1 + m + 6m + n + 1 = O(m + n)$$

vertices.

There are m edges from s to the driver vertices, at most $6m$ edges from driver vertices to driver-shift vertices, at most mn edges from driver-shift vertices to job vertices, and n edges from job vertices to t . Therefore,

$$E = O(m + 6m + mn + n) = O(mn)$$

in the worst case.

Using Ford–Fulkerson, the running time is $O(E|f^*|)$, where $|f^*|$ is the value of the maximum flow. Because each of the m drivers can be assigned to at most two jobs,

$$|f^*| \leq 2m.$$

Therefore, the maximum-flow computation takes

$$O(E|f^*|) = O(mn \cdot m) = O(m^2n).$$

The construction of the graph and extraction of the assignment take $O(mn)$ time, so the total running time is $O(m^2n)$. If $N = m + n$, then $m^2n \leq N^3$, so the running time is also $O(N^3)$.

Proof of Correctness

We prove that the algorithm outputs a valid assignment if and only if a valid assignment exists.

Forward direction. Suppose the algorithm outputs an assignment. It does so only when the maximum flow has value

$$\sum_{i=1}^n d_i.$$

Because every capacity in the network is an integer, the integrality theorem for maximum flow guarantees that an integral maximum flow exists. We can therefore interpret each unit of flow as a single driver-job assignment.

Every unit of flow follows a path of the form

$$s \rightarrow D_j \rightarrow K_{j,\ell} \rightarrow J_i \rightarrow t.$$

Such a path represents assigning driver j to job i . This assignment satisfies every constraint:

1. The edge $s \rightarrow D_j$ has capacity 2, so driver j is assigned to at most two jobs.
2. The edge $D_j \rightarrow K_{j,\ell}$ has capacity 1, so driver j is assigned to at most one job during shift ℓ .
3. An edge $K_{j,\ell} \rightarrow J_i$ exists only if job i occurs during shift ℓ and $i \in R_j$, so every driver is assigned only to a job they are eligible to perform.
4. The edge $J_i \rightarrow t$ has capacity d_i , so no more than d_i drivers are assigned to job i .
5. The total flow value is $\sum_i d_i$. Since the total capacity of all edges entering t is also $\sum_i d_i$, every edge $J_i \rightarrow t$ must be saturated. Therefore, exactly d_i drivers are assigned to every job i .

Thus, the algorithm's output is a valid assignment.

Reverse direction. Conversely, suppose a valid assignment exists. For every assignment of driver j to job i , let ℓ be the shift containing job i , and send one unit of flow along

$$s \rightarrow D_j \rightarrow K_{j,\ell} \rightarrow J_i \rightarrow t.$$

Because the assignment is valid, no driver is assigned to more than two jobs, so the capacity of $s \rightarrow D_j$ is respected. No driver performs more than one job during the same shift, so the capacity of $D_j \rightarrow K_{j,\ell}$ is respected. Each assigned job belongs to R_j and occurs during shift ℓ , so the required edge $K_{j,\ell} \rightarrow J_i$ exists. Finally, exactly d_i drivers are assigned to job i , so the capacity of $J_i \rightarrow t$ is respected.

Therefore, the valid assignment produces a feasible flow of value

$$\sum_{i=1}^n d_i.$$

No flow can have a greater value because the total capacity of the edges entering t is exactly $\sum_i d_i$. Consequently, the maximum-flow value is $\sum_i d_i$, and the algorithm outputs a valid assignment.

Hence, the algorithm outputs a valid assignment if and only if a valid assignment exists.